

FreshPilot - Back-Office pour les Revendeurs du Bateau de Thibault

Rapport de Projet

Realise par : Sivan COZZO & Alicia ABDOULAZIZE

Formation : Bac+4 - MTR

Date : Mars 2026

Sommaire

1. Contexte
 2. Design - Maquettes
 3. Architecture
 4. Modele Conceptuel de Donnees
 5. Backend
 6. Frontend
 7. Correspondance avec le cahier des charges
 8. Difficultes rencontrees
 9. Conclusion
-

1. Contexte

1.1 Presentation du projet

Le Bateau de Thibault est un fournisseur de produits de la mer qui distribue ses produits via un reseau de points relais repartis sur le territoire francais. Les gerants de ces points relais ont besoin d'un outil numerique leur permettant de gerer leur activite au quotidien.

FreshPilot est l'application web de type back-office que nous avons developpee pour reprendre a ce besoin. Elle permet aux gerants de :

- Visualiser et modifier les stocks de produits par categorie (poissons, crustaces, coquillages, etc.)
- Enregistrer les mouvements de stock : achats aupres du fournisseur, ventes aux clients, et pertes par peremption
- Modifier les prix et les promotions de facon individuelle ou groupee
- Consulter un tableau de bord avec des indicateurs financiers : chiffre d'affaires, marge, impot previsionnel
- Filtrer les donnees par periode, categorie et taux de promotion

1.2 Perimetre fonctionnel

Le projet couvre deux grandes parties conformement au cahier des charges :

Partie 1 - Gestion des produits : Affichage des produits par categorie, modification du stock et des promotions, envoi simultane de modifications pour plusieurs produits, gestion des erreurs de saisie.

Partie 2 - Donnees a historique : Enregistrement comptable des mouvements de stock (achat/vente/perte avec prix associes), calcul du chiffre d'affaires filtrable, calcul de la marge et de l'impot previsionnel, alertes visuelles.

2. Design - Maquettes

Avant de developper l'application, nous avons concu des maquettes pour definir l'experience utilisateur et valider le parcours de navigation.

2.1 Page de connexion

[Insérer la maquette de la page de login]

La page de connexion presente une interface epuree avec le logo FreshPilot, un formulaire identifiant/mot de passe, et un lien vers l'inscription. Le design utilise la couleur verte (#7BC67E) comme couleur principale pour vehiculer une image de fraicheur en coherence avec les produits de la mer.

2.2 Page d'inscription

[Insérer la maquette de la page d'inscription]

La page d'inscription reprend le meme design que la connexion avec les champs necessaires a la creation de compte : identifiant, mot de passe et confirmation du mot de passe.

2.3 Dashboard - Vue Produits

[Insérer la maquette du tableau produits]

La vue produits affiche un tableau interactif avec des badges de categorie en haut pour filtrer. Chaque colonne editable dispose d'un bouton crayon pour ouvrir le mode edition inline. Les boutons "Soumettre" et "Cancel" en bas permettent d'envoyer ou d'annuler les modifications.

2.4 Dashboard - Vue Donnees

[Insérer la maquette du dashboard KPIs]

La vue donnees presente les KPIs (chiffre d'affaires, marge, impot) avec des graphiques en courbes et des indicateurs d'evolution. Une barre de filtres en haut permet de selectionner la periode, les categories et la fourchette de promotion.

2.5 Barre laterale

[Insérer la maquette de la sidebar]

La barre laterale affiche les informations de l'utilisateur connecté (carte utilisateur) et la liste des membres de l'équipe du point relais (carte équipe).

3. Architecture

3.1 Architecture globale

[Insérer le schema d'architecture globale]

L'application suit une architecture client-serveur en trois couches : le frontend Angular communique avec le backend Django REST Framework via une API REST (HTTP/JSON), et le backend interagit avec la base de données MySQL.

3.2 Stack technique

Composant	Technologie	Version
Frontend	Angular (Standalone Components)	19.2.0
Langage frontend	TypeScript	5.7.2
CSS	Tailwind CSS	3.4.19
Backend	Django + Django REST Framework	6.0.2 / 3.14.0
Base de données	MySQL	8.x
Authentification	JWT (Simple JWT)	5.4.0
Graphiques	Chart.js	4.5.1
Decodage JWT cote client	jwt-decode	4.0.0
Gestion CORS	django-cors-headers	4.6.0

3.3 Justification des choix

Angular 19 a été choisi pour ses fonctionnalités modernes : les Standalone Components simplifient l'architecture en supprimant les NgModules, les Signals offrent une gestion réactive de l'état sans librairie externe, et le lazy loading via loadComponent optimise le chargement des pages.

Django REST Framework offre une API REST robuste avec des ViewSets générant automatiquement les opérations CRUD, un système de serializers pour transformer les données, et une intégration native avec Simple JWT pour l'authentification.

Tailwind CSS a été préféré à Angular Material pour sa flexibilité et la possibilité de créer un design sur mesure sans contrainte de composants prédéfinis.

MySQL a été choisi comme base de données relationnelle pour sa fiabilité et sa compatibilité avec Django.

4. Modele Conceptuel de Donnees

4.1 Schema MCD

[Insérer l'image du MCD]

Le schema de donnees comprend 9 modeles organisees autour de la chaine de distribution des produits de la mer, depuis le fournisseur jusqu'a la vente au client final.

4.2 Description des entites

Retailer - Represente un point de vente physique (ex: "FreshPoint Brest"). Contient le nom et l'adresse du point relais.

Utilisateur - Modele d'authentification personnalise etendant AbstractUser de Django. Chaque utilisateur est rattache a un point relais et possede un role (ROLE_ADMIN ou ROLE_USER).

Category - Classification des produits en 10 categories : Poissons, Crustaces, Coquillages, Fruits de mer, Algues, Cephalopodes, Poissons fumes, Conserves, Surgeles, Sauces et accompagnements.

Product - Produit generique dans le catalogue (ex: "Bar de ligne", "Homard breton"). Lie a une categorie.

Manufacturer - Fournisseur ou grossiste approvisionnant les points relais.

ManufacturerArticle - Variante d'un produit proposee par un fournisseur specifique, avec son unite de vente (kg, piece, lot, barquette, douzaine), sa disponibilite, son compteur de ventes cumule et un commentaire.

RetailerArticle - Article concret en vente dans un point relais. C'est le modele central de l'application. Il contient le prix de vente unitaire, le pourcentage de promotion en cours, le stock actuel et un indicateur d'archivage (suppression logique).

Purchase - Enregistrement d'un achat de lot aupres du fournisseur, avec la date, le cout total et la quantite.

Sale - Enregistrement d'une vente ou d'une perte, avec la date, le montant encaisse (0 en cas de peremption), la quantite et le pourcentage de promotion applique au moment de la transaction (discount_at_sale).

4.3 Choix de conception

La separation entre ManufacturerArticle et RetailerArticle permet de gerer le multi-site : un meme produit peut etre vendu dans plusieurs points relais avec des prix et des promotions differents.

Le champ discount_at_sale sur la table Sale capture le pourcentage de promotion au moment de la transaction, ce qui permet d'historiser les conditions de vente meme si la promotion est modifiee par la suite. Ce champ a ete ajoute via une seconde migration (0002_sale_discount_at_sale.py).

5. Backend

5.1 Models

Les 9 modeles Django sont definis dans app/models.py. Le modele Utilisateur etend AbstractUser pour ajouter les champs role et retailer. Les relations entre modeles utilisent des ForeignKey avec CASCADE pour garantir l'integrite referentielle.

```
class Utilisateur(AbstractUser):
    role = models.CharField(max_length=100, default='ROLE_USER')
    last_modification = models.DateField(auto_now=True)
    retailer = models.ForeignKey(Retailer, on_delete=models.CASCADE,
                                related_name='utilisateur', null=True)

class RetailerArticle(models.Model):
    discount = models.IntegerField(default=0)
    unit_price = models.IntegerField(default=0)
    quantity = models.IntegerField(default=0)
    is_archived = models.BooleanField(default=False)
    tig = models.ForeignKey(ManufacturerArticle, on_delete=models.CASCADE)
    retail = models.ForeignKey(Retailer, on_delete=models.CASCADE)
```

5.2 Serializers

Nous avons implemente 12 serializers. Deux meritent une attention particuliere :

RetailerArticleFlatSerializer aplatit les relations imbriquees pour fournir au frontend une structure simple et directement exploitable. Il mappe les champs du backend vers l'interface frontend : unit_price devient price, discount devient discount_percent, quantity devient stock, et les champs imbriques comme tig.prod.name deviennent simplement name.

```
class RetailerArticleFlatSerializer(serializers.ModelSerializer):
    name = serializers.SerializerMethodField() # tig.prod.name
    category = serializers.SerializerMethodField() # tig.prod.category.name
    price = serializers.IntegerField(source='unit_price')
    discount_percent = serializers.IntegerField(source='discount')
    stock = serializers.IntegerField(source='quantity')
    sales = serializers.SerializerMethodField() # tig.sales
    comment = serializers.SerializerMethodField() # tig.comments
```

CustomTokenObtainPairSerializer enrichit le JWT avec des claims personnalisés (username, retailer, role, etc.) pour éviter des appels API supplémentaires côté frontend.

```
class CustomTokenObtainPairSerializer(TokenObtainPairSerializer):
    @classmethod
    def get_token(cls, user):
        token = super().get_token(user)
        token['username'] = user.username
```

```

token['retailer'] = user.retailer_id
token['role'] = user.role
# ... autres claims
return token

```

5.3 Views et Endpoints

Le backend expose 10 ViewSets et 2 vues custom via les endpoints suivants :

Methode	Endpoint	Description
POST	/api/auth/register/	Inscription utilisateur
POST	/api/auth/login/	Connexion (obtention JWT)
POST	/api/auth/refresh/	Rafraichissement du token
GET	/api/retailer-articles/?retail=X	Articles d'un point relais
PATCH	/api/retailer-articles/{id}/	Modification article (prix, promo)
POST	/api/retailer-articles/submit-changes/	Modifications de stock en batch
GET	/api/purchases/?retail=X	Historique des achats
GET	/api/sales/?retail=X	Historique des ventes
GET	/api/utilisateurs/?retailer=X	Utilisateurs d'un point relais
GET	/api/categories/	Liste des categories

Le RetailerArticleViewSet utilise `get_serializer_class()` pour renvoyer le serializer aplati en lecture (GET) et le serializer imbrique en ecriture (POST/PATCH).

5.4 Endpoint submit-changes

L'endpoint le plus important du backend est POST `/api/retailer-articles/submit-changes/`. Il recoit une liste de modifications de stock et les traite de facon atomique :

Format de la requete :

```

[
  { "id": 12, "quantity_change": 100, "is_expired": false, "purchase_price": 15 },
  { "id": 7, "quantity_change": -27, "is_expired": false },
  { "id": 3, "quantity_change": -20, "is_expired": true }
]

```

Logique de traitement :

Cas	quantity_change	is_expired	Action
Achat	> 0	-	Stock += N, creation Purchase (cout = N x prix_achat)
Vente	< 0	false	Stock -= N, creation Sale (montant = N x prix_vente), compteur ventes += N
Perte	< 0	true	Stock -= N, creation Sale (montant = 0)

Les garanties techniques sont assurees par `transaction.atomic()` (toutes les modifications passent ou aucune) et `select_for_update()` (verrouillage de la ligne pour eviter les conflits en cas d'accès concurrent).

```
with transaction.atomic():
    for item in changes:
        article = RetailerArticle.objects.select_for_update().get(id=article_id)
        if quantity_change > 0:
            article.quantity += quantity_change
            Purchase.objects.create(retailer_article=article,
                                   quantity=quantity_change, total=quantity_change * purchase_price)
        elif not is_expired:
            article.quantity += quantity_change
            Sale.objects.create(retailer_article=article,
                               quantity=abs(quantity_change),
                               total=abs(quantity_change) * article.unit_price,
                               discount_at_sale=article.discount)
        else:
            article.quantity += quantity_change
            Sale.objects.create(retailer_article=article,
                               quantity=abs(quantity_change), total=0)
```

5.5 Authentification JWT

L'authentification repose sur Simple JWT avec les parametres suivants : duree du token d'accès de 60 minutes, duree du token de rafraichissement de 1 jour, et mise a jour automatique de `last_login` a chaque connexion.

Le JWT contient des claims personnalisés permettant au frontend d'identifier l'utilisateur et son point relais sans appel API supplémentaire :

```
{
  "user_id": 5,
  "username": "sivan.cozzo",
```

```

"first_name": "Sivan",
"last_name": "COZZO",
"retailer": 2,
"role": "ROLE_ADMIN",
"exp": 1711036200
}

```

Le flux d'inscription cree l'utilisateur et retourne immediatement un couple de tokens JWT, permettant une redirection directe vers le dashboard sans passer par la page de connexion.

5.6 Jeu de donnees (Seeder)

Un script de peuplement (python manage.py seed) genere un jeu de donnees realiste pour les tests et la demonstration :

Entite	Quantite
Points relais	15 (repartis dans toute la France)
Utilisateurs	43 (1 a 5 par point relais, mix admin/user)
Categories	10
Produits	60
Fournisseurs	15
Articles fournisseurs	~120-180
Articles point relais	~250-500
Achats	600
Ventes	1500 (dont 5% de pertes)

Les donnees sont distribuees de facon ponderee dans le temps (15% la derniere semaine, 25% le dernier mois, 25% le dernier trimestre, 35% l'annee) pour que les graphiques du dashboard affichent des courbes significatives.

6. Frontend

6.1 Routage et Guards

L'application utilise le lazy loading pour charger les pages a la demande :

Route	Composant	Protection
/	Redirection vers /dashboard	-
/login	LoginPageComponent	Public
/register	RegisterPageComponent	Public
/dashboard	DashboardPageComponent	authGuard
/add-product	AddProductPageComponent	authGuard
/validate-product	ValidateProductPageComponent	authGuard

Le authGuard verifie la validite du token JWT (presence et non-expiration du champ exp). En cas d'echec, l'utilisateur est redirige vers /login avec un parametre returnUrl pour revenir a la page souhaitee apres connexion.

```
export const authGuard: CanActivateFn = (route, state) => {
  const auth = inject(AuthService);
  const router = inject(Router);
  if (auth.isAuthenticated()) return true;
  router.navigate(['/login'], { queryParams: { returnUrl: state.url } });
  return false;
};
```

6.2 Services

AuthService gere le cycle de vie de la session utilisateur : connexion (appel API + stockage des tokens dans localStorage + decodage du JWT), deconnexion (suppression des tokens), verification d'authentification (comparaison de exp avec l'heure actuelle), et extraction du retailerId depuis le payload. L'etat utilisateur est gere via un Signal Angular pour une reactivite automatique.

```
currentUser = signal<JwtPayload | null>(null);

isAuthenticated(): boolean {
  const user = this.currentUser();
  if (!user) return false;
  return user.exp * 1000 > Date.now();
}
```

ClientService centralise tous les appels API REST avec gestion d'erreurs (catchError) et donnees de fallback. Methodes principales : getAllRetailArticles, submitChanges, updateRetailArticle, archiveArticle, getPurchases, getSales, getTeamMembers.

ProductsService transforme les donnees brutes de l'API en format d'affichage (RetailArticleDisplay) en calculant le prix en promotion et en formatant les valeurs avec les unites.

6.3 Composant Produits (ProductsComponent)

C'est le composant central de la Partie 1 du cahier des charges.

Affichage : Un tableau interactif affiche les articles du point relais avec les colonnes : Nom, Prix, Prix Promo, % Promo, Stock, Nb Vendus, Commentaire. Des badges de categorie en haut permettent de filtrer. Le prix en promotion est calcule automatiquement.

Modification inline : Un bouton crayon sur les colonnes Prix, % Promo et Stock ouvre une colonne d'edition. Pour le stock, un champ numerique accepte les valeurs positives (achat) ou negatives (vente/perte). Une checkbox "Perime ?" permet de marquer un retrait comme perte.

Envoi simultane : Le bouton “Soumettre” collecte toutes les modifications. Si des achats sont detectes, une modale s’ouvre pour saisir le prix d’achat unitaire. Les modifications sont envoyees en batch via submit-changes. Apres soumission, le tableau se recharge automatiquement.

Gestion des erreurs : Si le stock resultant est negatif, la ligne passe en rouge avec un message “Stock insuffisant”. La soumission est bloquee tant que des erreurs existent.

L’etat du composant utilise les Signals et Computed d’Angular 19 :

```
retailArticles = signal<RetailArticleDisplay[]>([]);
selectedCategory = signal<string | null>(null);
filteredArticles = computed(() => {
  const selected = this.selectedCategory();
  if (!selected) return this.retailArticles();
  return this.retailArticles().filter(a => a.category === selected);
});
```

6.4 Composant Donnees (DataComponent)

C’est le composant central de la Partie 2 du cahier des charges.

Filtres : Periode (Semaine, Mois, Trimestre, Annee), Type/Categorie (dropdown multi-selection avec checkboxes), Promotions (plage min/max du pourcentage).

KPIs affichees :

- **Chiffre d’affaires :** somme des totaux des ventes (hors pertes ou total = 0), avec pourcentage d’evolution par rapport a la periode precedente
- **Marge :** chiffre d’affaires moins la somme des achats, avec pourcentage d’evolution
- **Impot previsionnel :** 30% du benefice (si marge positive)

Graphiques Chart.js : Deux courbes lissees avec remplissage sous la courbe : evolution du CA par jour et evolution de la marge par jour sur la periode selectionnee.

Indicateurs visuels : Fleche verte et pourcentage vert pour une evolution positive, fleche rouge et pourcentage rouge pour une evolution negative.

Pour calculer l’evolution, le systeme compare la periode courante avec la periode immediatement precedente de meme duree :

```
private getPreviousDateRange(period: Period): { start: Date; end: Date } {
  const current = this.getDateRange(period);
  const duration = current.end.getTime() - current.start.getTime();
  return {
    start: new Date(current.start.getTime() - duration),
    end: new Date(current.start.getTime())
  };
}
```

6.5 Composants partages

UserCardComponent affiche les informations de l'utilisateur connecte (nom, role, email). **TeamCardComponent** affiche la liste des membres de l'equipe du point relais. **AppHeaderComponent** contient le nom de l'utilisateur et un bouton de deconnexion. Des versions mobile de ces composants assurent le responsive design.

7. Correspondance avec le cahier des charges

Partie 1 - Details et modification simultanees de plusieurs produits

Exigence	Statut
1) Affichage des produits par categorie avec nom, prix, prix promo, % promo, stock, vendus, commentaires	Implemente
2) Modifier le stock d'un produit avec bouton envoyer	Implemente
3) Modifier le % de promotion avec bouton envoyer	Implemente
4) Changement simultane de plusieurs produits via un bouton commun	Implemente
5) Gestion des erreurs : champ en rouge + message d'erreur	Implemente

Partie 2 - Donnees a historique

Exigence	Statut
1) Modification du stock enrichie : prix d'achat (ajout), prix de vente (vente), 0 (invendus)	Implemente
2) Calcul du CA filtrable par periode, categorie et promotion	Implemente
3) Resultat comptable (marge) et impot sur les societes (30% du benefice)	Implemente
4) Alerte automatique pour marge negative (visuel rouge)	Partiellement implemente

L'indicateur visuel rouge pour les evolutions negatives est implemente. Les alertes visuelles avancees (confettis lors de performances exceptionnelles) n'ont pas ete implementees dans cette version.

8. Difficultes rencontrees

8.1 JWT personnalise

Le token JWT standard de Simple JWT ne contient que l'ID utilisateur. Or, le frontend a besoin du `retailer_id` pour filtrer les donnees sans appel API supplementaire. Nous avons cree un `CustomTokenObtainPairSerializer` ajoutant des claims personnalises directement dans le payload du token, et utilise `jwt-decode` cote frontend pour les extraire.

8.2 Serializer conditionnel lecture/ecriture

Le format aplati necessaire au frontend est incompatible avec le format imbrique attendu pour les operations d'ecriture. Nous avons utilise `get_serializer_class()` dans le `ViewSet` pour renvoyer `RetailerArticleFlatSerializer` en lecture (GET) et `RetailerArticleSerializer` en ecriture (POST/PATCH).

8.3 Historisation du pourcentage de promotion

Quand une vente est enregistree et que la promotion change ensuite, il est impossible de savoir quel pourcentage etait en vigueur. Nous avons ajoute le champ `discount_at_sale` dans le modele `Sale` (migration 0002), capture automatiquement depuis `article.discount` au moment de la vente.

8.4 Modifications simultanees multi-types

L'utilisateur peut modifier le stock, le prix et la promotion de plusieurs articles en meme temps. Nous avons separe les flux : les modifications de prix/promotion sont envoyees via des PATCH individuels parallelises avec `forkJoin`, et les modifications de stock sont envoyees en batch via `submit-changes` avec transaction atomique.

9. Conclusion

Bilan

Ce projet a permis de developper une application web fonctionnelle repondant aux exigences du cahier des charges :

- Gestion complete des stocks avec enregistrement comptable (achats, ventes, pertes)
- Modification simultanee de plusieurs produits en une seule soumission
- Dashboard analytique avec KPIs financiers et graphiques interactifs
- Authentification securisee par JWT avec isolation des donnees par point relais
- Validation des saisies avec retour visuel en cas d'erreur

Competences acquises

- Developpement full-stack avec Angular 19 et Django REST Framework

- Gestion de l'authentification JWT avec claims personnalisées
- Conception d'une API REST avec serializers avancées
- Utilisation des Signals Angular pour la gestion réactive de l'état
- Transactions atomiques et verrouillage en base de données
- Visualisation de données avec Chart.js

Perspectives d'évolution

- Alertes visuelles avancées : notifications en cas de marge négative, confettis lors de performances exceptionnelles
- Export de rapports en PDF/CSV pour les bilans comptables
- Interface d'administration différenciée pour les ROLE_ADMIN
- Déploiement en production avec HTTPS, Nginx et Docker